AI Assisted Coding Lab 1

Rida Shireen

2303A52224

Batch – 42

Question 1:

Task 1: AI-Generated Logic Without Modularization (Fibonacci Sequence

Without Functions)

Prompt:

write a python code for printing fibonacci series upto n terms without using functions

Code:

LAB-1.py > ...

```python
# write a python code for printing fibonacci series upto n terms without using functions
n = int(input("Enter the number of terms: "))
a, b = 0, 1
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
print()
```

Output:

```
Enter the number of terms: 8
0 1 1 2 3 5 8 13
```

Justification:

This program prints the Fibonacci series upto n terms without using functions. All the logics are written in the main program, which makes it simple to understand. It works correctly for the given input. This approach is useful for small programs or quick testing. But it is a bit hard for larger numbers. The program accepts user input and prints the Fibonacci series upto the nth term.

Question 2:

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

Prompt:

write a optimized python code for printing fibonacci series up to n terms without using functions

Optimized Fibonacci series in Python

5m · +24 -3 · Auto

write a optimized python code for printing fibonacci series up to n terms without using functions

Agent Auto

Code:

LAB-1.py > ...

```python

# Optimized Fibonacci Series up to n terms (without functions)
n = int(input("Enter the number of terms: "))

# First two terms of Fibonacci series
a, b = 0, 1

# Print first term
if n >= 1:
    print(a, end=" ")

# Print second term
if n >= 2:
    print(b, end=" ")

# Generate and print remaining terms
for i in range(2, n):
    # Calculate next term
    c = a + b
    print(c, end=" ")
    # Update values for next iteration
    a, b = b, c

print()  # New line at the end
```

Output:

```
Enter the number of terms: 8
0 1 1 2 3 5 8 13
```

Justification:

This program prints the optimized code of Fibonacci series that removes extra variables and simplifies the logic. Compared to Task 1, it removes the unwanted and extra lines. It uses fewer lines of code and is easier to read. The program runs faster because unnecessary steps are avoided. The output remains correct after optimization. This makes the code more efficient and clean.

Question 3:

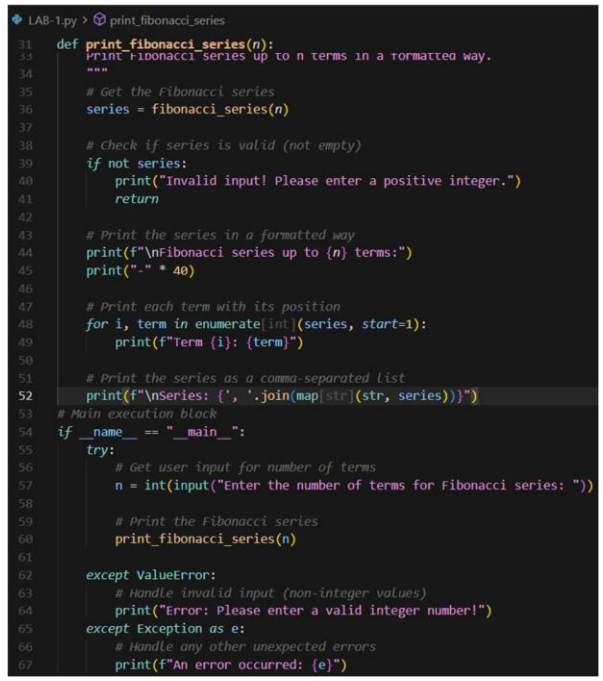
Task 3: Modular Design Using AI Assistance (Fibonacci Using Functions)

Prompt:

Write a python code for printing fibonacci series upto n terms and add meaningful comments

Code:

```
LAB-1.py > print_fibonacci_series
def fibonacci_series(n):
    """
    Generate and return Fibonacci series up to n terms.
    """
    # Handle edge cases
    if n <= 0:
        return []  # Return empty list for invalid input
    elif n == 1:
        return [0]  # First term is always 0
    elif n == 2:
        return [0, 1]  # First two terms are 0 and 1

    # Initialize the series with first two terms
    fib_series = [0, 1]

    # Generate remaining terms (n-2 terms, since we already have 2)
    # Each term is the sum of the previous two terms
    for i in range(2, n):
        next_term = fib_series[i - 1] + fib_series[i - 2]
        fib_series.append(next_term)

    return fib_series


def print_fibonacci_series(n):
    """
    Print Fibonacci series up to n terms in a formatted way.
    """
    # Get the Fibonacci series
    series = fibonacci_series(n)

    # Check if series is valid (not empty)
    if not series:
        print("Invalid input! Please enter a positive integer.")
        return
```



```
LAB-1.py > print_fibonacci_series
def print_fibonacci_series(n):
    """
    Print Fibonacci series up to n terms in a formatted way.
    """
    # Get the Fibonacci series
    series = fibonacci_series(n)

    # Check if series is valid (not empty)
    if not series:
        print("Invalid input! Please enter a positive integer.")
        return

    # Print the series in a formatted way
    print(f"\nFibonacci series up to {n} terms:")
    print("-" * 40)

    # Print each term with its position
    for i, term in enumerate(series, start=1):
        print(f"Term {i}: {term}")

    # Print the series as a comma-separated list
    print(f"\nSeries: {', '.join(map(str, series))}")
# Main execution block
if __name__ == "__main__":
    try:
        # Get user input for number of terms
        n = int(input("Enter the number of terms for Fibonacci series: "))

        # Print the Fibonacci series
        print_fibonacci_series(n)

    except ValueError:
        # Handle invalid input (non-integer values)
        print("Error: Please enter a valid integer number!")
    except Exception as e:
        # Handle any other unexpected errors
        print(f"An error occurred: {e}")
```

Output:

```
Enter the number of terms for Fibonacci series: 8

Fibonacci series up to 8 terms:
----------------------------------------
Term 1: 0
Term 2: 1
Term 3: 1
Term 4: 2
Term 5: 3
Term 6: 5
Term 7: 8
Term 8: 13

Series: 0, 1, 1, 2, 3, 5, 8, 13
```

Justifications:

In this task, the program prints the Fibonacci series with meaningful comments. The Fibonacci logic is written inside a user defined function. This makes the program modular and well organized. The function can be reused in different programs. Comments help in understanding the code easily. Debugging becomes simpler due to clear separation of logic. This approach follows good programming practices. It is suitable for large and real world applications.

Question 4

Task 4: Comparative Analysis – Procedural vs Modular Fibonacci Code

Prompt:

Procedular: write a python code for printing fibonacci series upto n terms without using functions

Modular: Write a python code for printing fibonacci series upto n terms and add meaningful comments

Code:

Modular:

LAB-1.py > print_fibonacci_series

```python
def fibonacci_series(n):
    """
    Generate and return Fibonacci series up to n terms.
    """
    # Handle edge cases
    if n <= 0:
        return []  # Return empty list for invalid input
    elif n == 1:
        return [0]  # First term is always 0
    elif n == 2:
        return [0, 1]  # First two terms are 0 and 1

    # Initialize the series with first two terms
    fib_series = [0, 1]

    # Generate remaining terms (n-2 terms, since we already have 2)
    # Each term is the sum of the previous two terms
    for i in range(2, n):
        next_term = fib_series[i - 1] + fib_series[i - 2]
        fib_series.append(next_term)

    return fib_series


def print_fibonacci_series(n):
    """
    Print Fibonacci series up to n terms in a formatted way.
    """
    # Get the Fibonacci series
    series = fibonacci_series(n)

    # Check if series is valid (not empty)
    if not series:
        print("Invalid input! Please enter a positive integer.")
        return
```

LAB-1.py > print_fibonacci_series

```python
def print_fibonacci_series(n):
    Print Fibonacci series up to n terms in a formatted way.
    """
    # Get the Fibonacci series
    series = fibonacci_series(n)

    # Check if series is valid (not empty)
    if not series:
        print("Invalid input! Please enter a positive integer.")
        return

    # Print the series in a formatted way
    print(f"\nFibonacci series up to {n} terms:")
    print("-" * 40)

    # Print each term with its position
    for i, term in enumerate(series, start=1):
        print(f"Term {i}: {term}")

    # Print the series as a comma-separated list
    print(f"\nSeries: {', '.join(map(str, series))}")
# Main execution block
if __name__ == "__main__":
    try:
        # Get user input for number of terms
        n = int(input("Enter the number of terms for Fibonacci series: "))

        # Print the Fibonacci series
        print_fibonacci_series(n)

    except ValueError:
        # Handle invalid input (non-integer values)
        print("Error: Please enter a valid integer number!")
    except Exception as e:
        # Handle any other unexpected errors
        print(f"An error occurred: {e}")
```

Procedular:

```
LAB-1.py > ...
# write a python code for printing fibonacci series upto n terms without using functions
n = int(input("Enter the number of terms: "))
a, b = 0, 1
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
print()
```

Outputs:

Procedular:

```
Enter the number of terms: 8
0 1 1 2 3 5 8 13
```

Modular:

```
Enter the number of terms for Fibonacci series: 8

Fibonacci series up to 8 terms:
----------------------------------------
Term 1: 0
Term 2: 1
Term 3: 1
Term 4: 2
Term 5: 3
Term 6: 5
Term 7: 8
Term 8: 13

Series: 0, 1, 1, 2, 3, 5, 8, 13
```

Justification:

In this task, the previous codes are compared(Task1 and Task3). The procedural code is simple but not reusable. All logic is written in one place, making it less organized. The modular code separates logic using functions. Modular code is easier to read and understand. Debugging is simpler in modular programs. It supports reuse and scalability. Therefore, modular code is better for large systems.

Question 5:

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches for Fibonacci Series)

Prompts:

Iterative Approach: write a python code for printing fibonacci series upto n terms using iterative approach

Fibonacci series iterative Python code

Now · +38 -24 · Auto

write a python code for printing fibonacci series upto n terms using iterative approach

Recursive Approach: write a python code for printing fibonacci series upto n terms using recursion

Python Fibonacci series recursion

1m · +35 -24 · Auto

write a python code for printing fibonacci series upto n terms using recursion

Codes:

Iterative Approach:

LAB-1.py > fibonacci_iterative

```python
def fibonacci_iterative(n):
    if n <= 0:
        print("Please enter a positive number.")
        return

    # First two terms of Fibonacci series
    if n >= 1:
        print(0, end=" ")
    if n >= 2:
        print(1, end=" ")

    # Initialize first two terms
    a, b = 0, 1

    # Generate remaining terms
    for i in range(2, n):
        next_term = a + b
        print(next_term, end=" ")
        a, b = b, next_term

    print()  # New line after printing all terms


# Main program
if __name__ == "__main__":
    try:
        n = int(input("Enter the number of terms: "))
        print(f"Fibonacci series up to {n} terms:")
        fibonacci_iterative(n)
    except ValueError:
        print("Invalid input! Please enter a valid integer.")
```

Recursive Approach:

LAB-1.py > print_fibonacci_series

```python
def fibonacci(n):
    """
    Recursive function to calculate the nth Fibonacci number.
    Base cases: F(0) = 0, F(1) = 1
    Recursive case: F(n) = F(n-1) + F(n-2)
    """
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)


def print_fibonacci_series(n):
    """
    Prints the Fibonacci series up to n terms using recursion.
    """
    print(f"Fibonacci series up to {n} terms:")
    for i in range(1, n + 1):
        print(fibonacci(i), end=" ")
    print()  # New line after the series


# Main program
if __name__ == "__main__":
    try:
        n = int(input("Enter the number of terms: "))
        if n <= 0:
            print("Please enter a positive integer.")
        else:
            print_fibonacci_series(n)
    except ValueError:
        print("Invalid input! Please enter a valid integer.")
```

Outputs:

Iterative Approach:

```
Enter the number of terms: 8
Fibonacci series up to 8 terms:
0 1 1 2 3 5 8 13
```

Recursive Approach:

```
Enter the number of terms: 8
Fibonacci series up to 8 terms:
1 1 2 3 5 8 13 21
```

Justification:

The program prints the code by two approaches, Iterative and Recursive. The iterative approach uses a loop to generate Fibonacci numbers. It runs faster and uses less memory. This approach works well for large values of n. The recursive approach uses function calls repeatedly. It takes more time due to repeated calculations. It also uses more memory because of the call stack. Recursion should be avoided for large inputs.